

Progettazione backend

Mattia Cavina

03 febbraio 2026

1 Scelta del Framework

Dopo una analisi sui vari framwork disponibili in C# che potessero essere adatti al nostro progetto, abbiamo deciso di utilizzare Entity Framework Core per la gestione del database. La connesione al database è gestita da una classe Factory che si occupa di creare la connesione e restituire un oggetto DbConnection che viene utilizzato per la gestione delle chiamate al database.

2 Pro e Contro della Scelta

2.1 Pro

- **Integrazione con .NET:** Entity Framework Core è progettato per integrarsi perfettamente con l'ecosistema .NET, facilitando lo sviluppo di applicazioni backend.
- **Supporto per LINQ:** Permette di utilizzare LINQ (Language Integrated Query) per scrivere query in modo più intuitivo e leggibile.
- **Migrations:** Offre un sistema di migrazioni che semplifica la gestione delle modifiche al database nel tempo.
- **Cross-platform:** Essendo basato su .NET Core, è compatibile con diverse piattaforme, inclusi Windows, Linux e macOS.

- **Mappa oggetti-relazionale:** Facilita la mappatura tra le classi C# e le tabelle del database, riducendo la quantità di codice boilerplate necessario.

2.2 Contro

- **Performance:** In alcuni scenari, Entity Framework Core può essere meno performante rispetto a soluzioni più leggere o a query SQL scritte manualmente.
- **Curva di apprendimento:** Per chi non ha familiarità con i concetti di ORM (Object-Relational Mapping), potrebbe esserci una curva di apprendimento iniziale.
- **Controllo limitato:** A volte, l'astrazione offerta da Entity Framework Core può limitare il controllo sulle query generate, rendendo difficile ottimizzare le prestazioni in casi complessi.
- **Overhead:** L'uso di un ORM può introdurre un overhead aggiuntivo rispetto a soluzioni più dirette, soprattutto in applicazioni ad alte prestazioni.

2.3 Soluzioni ai Contro

Per mitigare i contro di Entity Framework Core, abbiamo adottato alcune strategie:

- **Ottimizzazione delle query:** Utilizziamo strumenti di profiling per identificare e ottimizzare le query lente, e in alcuni casi scriviamo query SQL personalizzate quando necessario.
- **Utilizzo di caching:** Implementiamo meccanismi di caching per ridurre il numero di chiamate al database e migliorare le prestazioni complessive dell'applicazione.

Queste soluzioni verranno implementate al bisogno durante lo sviluppo del progetto, garantendo un equilibrio tra facilità d'uso e prestazioni.

3 Struttura

La parte del backend verrà sviluppata in un modulo separato esponendo un'API RESTful che consente al frontend di interagire con il database. La gestione delle query verrà incapsulata in metodi di DAO dedicati che si occuperanno di eseguire le operazioni CRUD (Create, Read, Update, Delete) sui dati. Questo incapsulamento permetterà l'ottimizzazione delle query e la gestione centralizzata delle operazioni sul database, facilitando la manutenzione e l'evoluzione del codice nel tempo.

3.1 Gestione della Sicurezza dei Dati

La sicurezza dei dati verrà garantita attraverso un isolamento degli oggetti. Le classi di accesso ai dati creati dall'ORM saranno utilizzati esclusivamente per la gestione delle query e non esposti direttamente al frontend. Al frontend verranno esposti solo oggetti di trasferimento dati (DTO) che contengono solo le informazioni necessarie per l'interazione con l'utente, evitando di esporre dettagli sensibili o strutture interne del database.

4 Implementazione della mappatura

Per mappare gli oggetti del database generati automaticamente mediante NET in classi di dominio, utilizzeremo la libreria Mapster. Mapster è una libreria di mappatura leggera e ad alte prestazioni che consente di mappare facilmente tra oggetti di diversi tipi, come ad esempio tra entità del database e DTO (Data Transfer Objects). Utilizzando Mapster, possiamo definire le regole di mappatura in modo semplice e flessibile, consentendo una conversione efficiente tra le classi generate da Entity Framework Core e le classi di dominio utilizzate nel nostro backend. Questa conversione è resa possibile mediante la classe (MapsterConfig), implementazione dell'interfaccia (IRegister) di Mapster, che definisce le regole di mappatura tra le classi generate e le classi di dominio. Possiamo quindi configurare la profondità della mappatura per gestire correttamente le relazioni tra le entità, evitando problemi di loop infiniti o di prestazioni.

5 Creazione delle Interfacce di Accesso ai Dati

Si è scelto di creare dei Repository divisi per "Argomento" (ad esempio, un repository per la gestione degli utenti, uno per la gestione dei prodotti, ecc.) che si occuperanno di incapsulare le operazioni di accesso ai dati. Questi repository sono stati generati dopo un paio di classi di test per verificare la correttezza delle query e l'efficacia della mappatura tra le classi generate da Entity Framework Core e le classi di dominio. Queste ultime sono state provate mediante unit test per garantire la correttezza delle operazioni di accesso ai dati e la coerenza tra le classi generate e le classi di dominio. Le interfacce generate sono:

- **IBankRepository**: per la gestione delle operazioni relative alle banche.
- **ICustomerRepository**: per la gestione delle operazioni relative ai clienti
- **IDocumentRepository**: per la gestione delle operazioni relative ai documenti, inclusa la gestione delle righe dei documenti, il loro versionamento e la gestione della promozione dei documenti da offerta a ordine.
- **ILanguageRepository**: per la gestione delle operazioni relative alle lingue supportate dall'applicazione.
- **ILookupRepository**: per la gestione delle operazioni relative alle tabelle di lookup, che contengono dati di riferimento utilizzati in diverse parti dell'applicazione. Le operazioni all'interno saranno di sola lettura, in quanto queste tabelle vengono gestite direttamente dagli amministratori del database e non devono essere modificate dall'applicazione.
- **IMessageRepository**: per la gestione delle operazioni relative ai messaggi, ovvero notifiche e feedback che vengono inviati agli utenti in risposta a determinate azioni o eventi all'interno dell'applicazione.
- **IMountingRepository**: per la gestione delle operazioni relative ai montaggi degli impianti, inclusa la gestione degli operatori coinvolti.
- **IParagraphRepository**: per la gestione delle operazioni relative ai paragrafi, che rappresentano le sezioni o i blocchi di testo all'interno dei documenti.

- **IProductRepository**: per la gestione delle operazioni relative ai prodotti, inclusa la gestione delle loro distinte basi.
- **IPropertyRepository**: per la gestione delle operazioni relative alle proprietà, che rappresentano le caratteristiche o gli attributi dei prodotti.
- **ISellConditionRepository**: per la gestione delle operazioni relative alle condizioni di vendita, che rappresentano le regole o i criteri applicati alle offerte e agli ordini.
- **IShipperRepository**: per la gestione delle operazioni relative ai spedizionieri.
- **IStaticRepository**: per la gestione delle operazioni relative ai dati statici, che rappresentano informazioni di riferimento che non cambiano frequentemente, come ad esempio Nazioni, Zone Geografiche, ecc.
- **IUserRepository**: per la gestione delle operazioni relative agli utenti, inclusa la gestione dei loro ruoli e permessi all'interno dell'applicazione.

5.1 Impletazione Repository

I repository sopra citati sono stati sviluppati con un sistema test-driven, creando prima i test per verificare la correttezza delle operazioni di accesso ai dati e la coerenza tra le classi generate da Entity Framework Core e le classi di dominio, e successivamente implementando i metodi necessari per far passare i test. Questa metodologia ha permesso di garantire la qualità del codice e la correttezza delle operazioni di accesso ai dati fin dalle prime fasi dello sviluppo, riducendo il rischio di bug e problemi di coerenza tra le classi generate e le classi di dominio. Dato la continua evoluzione del progetto avere una suite di test ben definita ci ha permesso di implementare nuove funzionalità e modificare quelle esistenti con maggiore sicurezza. Nel corso dello sviluppo delle operazioni CRUD si è reso necessario implementare già in questa fase le tabelle di backup per garantire il recupero dei dati storici.

6 Tabelle Backup

Le tabelle di backup non sono per tutte le tabelle del database, ma solo per quelle significative come ad esempio la tabelle dei prodotti o dei documenti, che rappresentano le entità principali del nostro sistema e che subiscono frequenti modifiche. Esse sono state pensate con una logica ben definita:

- **Struttura simile alla tabella originale:** Le tabelle di backup hanno una struttura simile a quella delle tabelle originali, con l'aggiunta di campi per tracciare la data e l'ora della modifica, l'utente che ha effettuato la modifica e un campo per indicare se la riga è stata eliminata o meno.
- **Gestione delle modifiche:** Ad ogni operazione di update sulla tabella originale, scatta un trigger before update che copia la riga originale nella tabella di backup, mantenendo traccia delle modifiche effettuate e permettendo di recuperare lo stato precedente dei dati in caso di errori o necessità di analisi storiche.
- **Recupero dei dati storici:** In caso di necessità, è possibile recuperare i dati storici tramite l'id della riga ed ordinarle per "LastEditDate" per ottenere la cronologia delle modifiche effettuate su quella riga specifica.
- **Restore dei dati:** In caso di errori o modifiche indesiderate, è possibile utilizzare le tabelle di backup per ripristinare i dati allo stato precedente, garantendo la sicurezza e l'integrità dei dati nel tempo. Verrà comunque incrementata la data di ultima modifica per tenere traccia del momento in cui è stato effettuato il restore e dell'utente che ha effettuato l'operazione.

6.1 Trigger

I trigger come le tabelle di backup sono stati gestiti inserendoli nel codice di SSIS per garantire la loro creazione automatica durante la fase di deploy del database. In questo modo, ogni volta che viene effettuato un deploy del database, i trigger vengono creati automaticamente, garantendo la corretta gestione delle modifiche e il tracciamento delle operazioni sui dati fin dalle prime fasi di utilizzo del sistema.